

TB anatomy refresh

Before UVM classes and phases, every verification engineer meets the same classic picture

DUT, tb, reg, wire, initial

- The DUT is synthesizable RTL, the block you are verifying
- The testbench module is simulation-only
- Stimulus inputs are typically reg because the testbench assigns them in an initial or
- Outputs you observe are wire or logic nets driven by the DUT
- An initial block advances time with delays, prints with display, and stops with finish
- That five-part split

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / TB anatomy

INTERACTIVE TOOL

Testbench anatomy explorer

See how a Verilog **testbench** wraps a **DUT**: stimulus (reg/logic), observed nets (wire), instance ports, then \$display / \$finish. Starter: a 2-input AND with a tiny initial that prints y and ends simulation.

Starter example: and2 DUT with reg a,b / wire y, \$display of y, then \$finish. Click anatomy parts and Step the timeline.

Challenge 0 / 22 cleared

Quiz: DUT: DUT means...

☐ Design Under Test — the synthesizable (or RTL) module being verified

☐ Device Upload Tool

☐ only the \$finish statement

☐ the forever clock generator alone

Show hint

Check

Next

Load challenge setup

Idle

Quiz: DUT

Quiz: testbench

Quiz: reg in TB

Quiz: wire observe

Quiz: \$display

Quiz: \$finish

Quiz: synth

Quiz: logic

Quiz: who makes clk

Quiz: no \$finish

Load starter

Select DUT

Select stimulus

Inspect a

TB anatomy lab starter

Real testbench file

```
wsl - module03-tb-anatomy/examples/navigation

# Track A - real Unix

# real shell session (Track A - classic TB anatomy)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module03-tb-anatomy

$ ls examples/and2-tb/
and2_tb.v

$ cat examples/and2-tb/and2_tb.v
// and2_tb.v - classic TB anatomy (DUT vs tb, reg/wire, initial)
module and2(input a, b, output y);
    assign y = a & b;
endmodule

module tb;
    reg a, b;      // stimulus - TB drives these
    wire y;        // observe - DUT drives this net

    and2 dut(.a(a), .b(b), .y(y));

    initial begin
        a = 0; b = 0;
        #10 a = 1;
        #10 b = 1;
        #10 $display("y=%b (expect 1)", y);
        #10 $finish;
    end
endmodule

$ iverilog -t null examples/and2-tb/and2_tb.v
# syntax check passed
```

Real shell — and2 testbench

Pitfalls to watch

- Do not drive a wire the DUT also drives, that creates a contention error
- Do not put synthesizable checks only in the testbench and forget what the RTL actually
- Do not skip finish in long runs unless you know the simulator will stop another way
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, step the timeline and finish a few challenges
- On real SystemVerilog
- When you are ready, take the short quiz, then continue to UVM phases in the next module